

Trabajo Práctico 3 – Sistemas de Computación



Integrantes:

- Brigido Tomás
- Siñuka Javier
- Tosolini Agustín

1	Introducción	3
2	Trabajo Práctico 1: Exploración del entorno UEFI y la Shell	4
2.1	Arranque en el entorno virtual	4
2.2	Exploración de Dispositivos (Handles y Protocolos)	4
2.3	Análisis de Variables Globales (NVRAM)	6
2.4	Footprinting de Memoria y Hardware	8
3	Trabajo Práctico 2: Desarrollo, compilación y análisis de seguridad	11
3.1	Desarrollo de la Aplicación	11
3.2	Compilación a Formato PE/COFF	12
3.3	Análisis de Metadatos y Decompilación	13
4	Trabajo Práctico 3: Ejecución en Hardware Físico (Bare Metal)	16
4.1	Preparación del medio de arranque (USB)	16
4.2	Configuración del BIOS/UEFI	17
4.3	Ejecución en Bare Metal	17
5	Conclusión	20
6	Conclusion – Respuestas solicitadas	21

1 Introducción

El presente informe documenta el desarrollo de las tres partes del trabajo práctico propuesto en el archivo `Arquitectura .pdf`, centradas en la exploración del entorno UEFI, la interacción con su shell, el análisis de variables y mapas de memoria, la construcción de una aplicación nativa y su posterior ejecución tanto en entorno virtual como en hardware físico.

En la primera parte se analiza cómo UEFI abstrae el hardware mediante handles y protocolos, cómo administra el arranque a través de variables persistentes en NVRAM y qué información expone sobre memoria, buses y controladores antes de cargar un sistema operativo. En la segunda parte se desarrolla una aplicación sencilla en C para ejecutarse como binario UEFI, se compila al formato PE/COFF requerido por el firmware y se inspecciona su comportamiento con herramientas de análisis estático. Finalmente, en la tercera parte se traslada el binario generado a un entorno bare metal, sorteando las restricciones de arranque seguro para validar su ejecución directa desde firmware.

Desde un punto de vista arquitectónico, UEFI reemplaza el modelo monolítico y fuertemente acoplado del BIOS por una interfaz estandarizada basada en servicios, estructuras de datos globales y protocolos publicados por el firmware. Esto permite que aplicaciones pre-boot, cargadores y utilidades de diagnóstico interactúen con consola, almacenamiento, variables persistentes y memoria de manera uniforme, independientemente del fabricante de la placa base o del dispositivo subyacente.

2 Trabajo Práctico 1:

Exploración del entorno UEFI y la Shell

2.1 Arranque en el entorno virtual

Para iniciar el laboratorio se utilizó QEMU junto con OVMF, que provee una implementación abierta del firmware UEFI. A diferencia de un arranque legado basado en BIOS, este entorno presenta una shell interactiva, servicios de arranque, servicios de tiempo de ejecución y una base de datos de protocolos que permiten inspeccionar el sistema antes de la carga del sistema operativo.

El arranque en un entorno virtual resulta útil porque permite repetir pruebas de forma controlada, observar el comportamiento del firmware sin depender de una máquina física y validar el flujo de trabajo de exploración y desarrollo en condiciones reproducibles.

En términos de ejecución, OVMF implementa las fases esperadas del firmware UEFI, incluyendo la inicialización de plataforma y la transición hacia la selección del dispositivo de arranque. La shell UEFI se ejecuta todavía dentro del contexto del firmware, por lo que los comandos disponibles interactúan directamente con las estructuras internas del boot environment y no con servicios de un sistema operativo generalista.

```
qemu-system-x86_64 -m 512 -bios /usr/share/ovmf/OVMF.fd -net none
```

2.2 Exploración de Dispositivos (Handles y Protocolos)

La exploración inicial se realizó con los comandos `map`, `ls` y `dh -b`. El primero enumera los sistemas de archivos y dispositivos que el firmware expone a la shell, mientras que `dh -b` permite inspeccionar los handles y los protocolos asociados a cada uno. Esto evidencia que UEFI trabaja sobre abstracciones de software en lugar de depender de direcciones o puertos de hardware fijos.

Un **handle** en UEFI puede entenderse como un contenedor opaco administrado por el firmware al que se le asocian uno o más protocolos. Cada protocolo define un contrato funcional, por ejemplo acceso a bloques, sistema de archivos simple, salida gráfica o consola de texto. En consecuencia, una aplicación no necesita conocer detalles eléctricos o registros específicos del dispositivo: le basta con localizar el protocolo adecuado y operar a través de su interfaz publicada.

```
1 Shell> map
2 Shell> FS0:
3 FS0:\> ls
4 Shell> dh -b
```

```
UEFI Interactive Shell v2.2
EDK II
UEFI v2.70 (Ubuntu distribution of EDK II, 0x00010000)
Mapping table
  FS0: Alias(s) :HD0a1::BLK1:
        PciRoot(0x0)/Pci(0x1,0x1)/Ata(0x0)/HD(1,MBR,0xBE1AFDFA,0x3F,0xFBFC1)
  BLK0: Alias(s) :
        PciRoot(0x0)/Pci(0x1,0x1)/Ata(0x0)
  BLK2: Alias(s) :
        PciRoot(0x0)/Pci(0x1,0x1)/Ata(0x0)
Press ESC in 1 seconds to skip startup.nsh or any other key to continue.
Shell> map
Mapping table
  FS0: Alias(s) :HD0a1::BLK1:
        PciRoot(0x0)/Pci(0x1,0x1)/Ata(0x0)/HD(1,MBR,0xBE1AFDFA,0x3F,0xFBFC1)
  BLK0: Alias(s) :
        PciRoot(0x0)/Pci(0x1,0x1)/Ata(0x0)
  BLK2: Alias(s) :
        PciRoot(0x0)/Pci(0x1,0x1)/Ata(0x0)
Shell> FS0:
FS0:\> ls
Directory of: FS0:\
05/01/2026  12:24          10,243  NuVars
05/01/2026  09:22             10  test.txt
          2 File(s)        10,253 bytes
          0 Dir(s)
```

2.1: output comandos map y ls

```

Shell> dh -b
Handle dump
01: LoadedImage (DxeCore)
02: Decompress
03: FirmwareVolume2 DevicePath(..F8EB-4F34-AAEA-3EE4AF6516A1)) FirmwareVolumeBlock
04: EE4E5898-3914-4259-9D6E-DC7BD79403CF
05: ImageDevicePath(..AD6B-4F3A-B60B-F59899003443)) LoadedImage (DevicePathDxe)
06: DevicePathFromText DevicePathToText DevicePathUtilities
07: ImageDevicePath(..87AB-47F9-A3FE-D50B76D89541)) LoadedImage (PcdDxe)
08: GetPcdInfo GetPcdInfoProtocol Pcd Pcd
09: BB00A5CA-08CE-462F-A537-43C74A825CA4 ImageDevicePath(..7BCE-40AF-B5BB-47E3611EB5C2)) LoadedImage
(TdxDxe)
0A: ImageDevicePath(..A7EB-4730-8C8E-CC466A9ECC3C)) LoadedImage (ReportStatusCodeRouterRuntimeDxe)
0B: SmartCardReader RscHandler
0C: ImageDevicePath(..43B7-4784-95B1-F4226CB40CEE)) LoadedImage (RuntimeDxe)
0D: RuntimeArch
0E: ImageDevicePath(..7FD6-4665-8646-88E33EF71DFC)) LoadedImage (SecurityStubDxe)
0F: SecurityArch Security2Arch
10: DeferredImageLoad
11: DebugSupport 96F46153-97A7-4793-ACC1-FA19BF78EA97 EBCInterpreter ImageDevicePath(..73D0-11D4-B0

```

2.2: output comando dh -b

Pregunta: Al ejecutar el comando map y dh, vemos protocolos e identificadores en lugar de puertos de hardware fijos. ¿Cuál es la ventaja de seguridad y compatibilidad de este modelo frente al antiguo BIOS?

La ventaja es que esto permite que el firmware sea independiente de la arquitectura del hardware. Al no permitir que las aplicaciones accedan directamente al hardware y forzarlas a usar interfaces (protocolos) controladas por el firmware, se reduce la superficie de ataque para código malicioso que intente manipular periféricos de forma directa o no autorizada.

Además, este modelo favorece la portabilidad del software de prearranque. Un mismo binario puede ejecutarse sobre distintas plataformas siempre que el firmware implemente los protocolos exigidos por la especificación. Esto desacopla la lógica de alto nivel de los detalles concretos del chipset, del controlador SATA/NVMe o del adaptador de video, algo que en BIOS tradicional solía resolverse con supuestos rígidos y llamadas específicas de plataforma.

2.3 Análisis de Variables Globales (NVRAM)

La inspección de variables persistentes permite observar cómo UEFI mantiene estado entre reinicios. A diferencia de variables de entorno efímeras, las variables de NVRAM son almacenadas por el firmware con atributos explícitos, como acceso durante servicios de arranque, acceso en tiempo de ejecución o persistencia no volátil. Esto las convierte en un mecanismo central tanto para la configuración del sistema como para la cadena de arranque.

Dentro de este conjunto, `BootOrder` y `Boot####` pertenecen normalmente al espacio de variables globales de EFI y participan en la fase BDS (Boot Device Selection). En esa fase, el firmware ya inicializó suficientes servicios como para poder enumerar dispositivos, consultar variables persistentes y decidir qué cargador EFI intentar ejecutar.

```
Variable NV+RT+BS 'EFIGlobalVariable:BootOrder' DataSize = 0x06
00000000: 00 00 01 00 02 00 *.....*
Variable NV+RT+BS 'EFIGlobalVariable:Boot0002' DataSize = 0x58
00000000: 01 00 00 00 2C 00 45 00-46 00 49 00 20 00 49 00 *.....E.F.I. .I.*
00000010: 6E 00 74 00 65 00 72 00-6E 00 61 00 6C 00 20 00 *n.t.e.r.n.a.l. .*
00000020: 53 00 68 00 65 00 6C 00-6C 00 00 00 04 07 14 00 *S.h.e.l.l.....*
00000030: C9 BD B8 7C EB F8 34 4F-AA EA 3E E4 AF 65 16 A1 *...l..40..>..e.*
00000040: 04 06 14 00 83 A5 04 7C-3E 9E 1C 4F AD 65 E0 52 *.....l>..O.e.R*
00000050: 68 D0 B4 D1 7F FF 04 00- *h.....*
Variable NV+RT+BS 'EFIGlobalVariable:Boot0001' DataSize = 0x6C
00000000: 01 00 00 00 1E 00 55 00-45 00 46 00 49 00 20 00 *.....U.E.F.I. .*
00000010: 51 00 45 00 4D 00 55 00-20 00 48 00 41 00 52 00 *Q.E.M.U. .H.A.R.*
00000020: 44 00 44 00 49 00 53 00-4B 00 20 00 51 00 4D 00 *D.D.I.S.K. .Q.M.*
00000030: 30 00 30 00 30 00 30 00-31 00 20 00 00 00 02 01 *0.0.0.0.1. ....*
00000040: 0C 00 D0 41 03 0A 00 00-00 00 01 01 06 00 01 01 *...A.....*
00000050: 03 01 08 00 00 00 00 00-7F FF 04 00 4E AC 08 81 *.....N...*
00000060: 11 9F 59 4D 85 0E E2 1A-52 2C 59 B2 *..YM...R,Y.*
```

2.3: output comando `dumpstore -b` parte 1

```
Variable NV+RT+BS 'EFIGlobalVariable:Boot0000' DataSize = 0x3E
00000000: 09 01 00 00 2C 00 55 00-69 00 41 00 70 00 70 00 *.....U.i.A.p.p.*
00000010: 00 00 04 07 14 00 C9 BD-B8 7C EB F8 34 4F AA EA *.....l..40..*
00000020: 3E E4 AF 65 16 A1 04 06-14 00 21 AA 2C 46 14 76 *>..e.....!.,F.u*
00000030: 03 45 83 6E 8A B6 F4 66-23 31 7F FF 04 00 *..E.n...f#1...*
Variable RT+BS 'EFIGlobalVariable:BootCurrent' DataSize = 0x02
00000000: 02 00 *..*
```

2.4: output comando `dumpstore -b` parte 2

```
Shell> set TestSeguridad "Hola UEFI"
Shell> set -v
    path = FS0:\efi\tools\;FS0:\efi\boot\;FS0:\
nonesting = False
debuglasterror = 0x0
lasterror = 0x0
profiles = ;Driver1;Debug1;network1;network2;
uefishellsupport = 3
uefishellversion = 2.2
uefiversion = 2.70
TestSeguridad = Hola UEFI
Shell> _
```

2.5: output comando `dumpstore set -v`

Pregunta: *Observando las variables `Boot####` y `BootOrder`, ¿cómo determina el Boot Manager la secuencia de arranque?*

El orden de ejecución lo define la variable global `BootOrder`. Esta contiene una lista de los números de las variables `Boot####`

Cada entrada `Boot####` representa una opción concreta de arranque y contiene metadatos como la descripción visible, el dispositivo asociado y la ruta del cargador EFI. El Boot Manager recorre la lista definida en `BootOrder` de izquierda a derecha e intenta ejecutar cada entrada en ese orden; si una opción falla o no está disponible, continúa con la siguiente. De esta manera, `BootOrder` funciona como la política de prioridad y `Boot####` como el conjunto de destinos posibles.

Técnicamente, cada entrada `Boot####` codifica una estructura con atributos, una cadena descriptiva y un `device path`, es decir, una representación estructurada del camino hacia el recurso de arranque. Ese `device path` puede apuntar a un disco, una partición del sistema EFI y un archivo `.efi` concreto. Por eso el firmware no solo decide “qué dispositivo” probar, sino también “qué ejecutable EFT” invocar dentro de ese dispositivo.

2.4 Footprinting de Memoria y Hardware

En esta etapa se relevaron tres aspectos distintos del entorno de prearranque. Con `memmap -b` se observaron las regiones de memoria clasificadas por tipo de servicio o uso; con `pci -b` se listaron dispositivos conectados al bus PCI; y con `drivers -b` se identificaron los controladores que el firmware ya había cargado.

```
1 Shell> memmap -b
2 Shell> pci -b
3 Shell> drivers -b
```

Este relevamiento es valioso desde el punto de vista de seguridad porque permite determinar qué superficies quedan disponibles antes del arranque del sistema operativo y qué componentes del firmware continúan teniendo influencia sobre la ejecución posterior.

El comando `memmap` es especialmente útil porque expone la clasificación que el propio firmware realiza sobre la RAM. Regiones como `LoaderCode`, `BootServicesCode`,

deben preservarse cuando el sistema operativo invoque `ExitBootServices` para abandonar el entorno de arranque.

```
BS_Data  000000001F147000-000000001F147FFF 00000000000000001 0000000000000000F
BS_Code  000000001F148000-000000001F14CFFF 00000000000000005 0000000000000000F
BS_Data  000000001F14D000-000000001F14ECFF 000000000000003A0 0000000000000000F
```

2.6: output comando memmap -b parte 1

```
BS_Data   :          9,680 Pages (39,649,280 Bytes)
RT_Code   :           256 Pages (1,048,576 Bytes)
RT_Data   :           581 Pages (2,379,776 Bytes)
```

2.7: output comando memmap -b parte 2

Pregunta: *En el mapa de memoria (memmap), existen regiones marcadas como `RuntimeServicesCode`. ¿Por qué estas áreas son un objetivo principal para los desarrolladores de malware (Bootkits)?*

Las áreas de `RuntimeServices` permanecen activas y residentes en la memoria RAM mientras la computadora está encendida. Si el malware se logra alojar en esas regiones puede interceptar llamadas del sistema operativo al firmware incluso después de que el sistema operativo haya cargado. Esto le otorga un nivel de persistencia y control extremadamente alto, permitiéndole sobrevivir a reinstalaciones del sistema operativo o manipular variables de seguridad de bajo nivel de forma invisible.

La diferencia crítica respecto de `BootServicesCode` es que esta última deja de ser válida una vez completada la transición al sistema operativo, mientras que `RuntimeServicesCode` sigue siendo necesaria para operaciones como lectura/escritura de variables EFI, consulta de tiempo o reinicios controlados. Por ello, cualquier modificación maliciosa en esa zona impacta directamente en la interfaz que el sistema operativo conserva con el firmware tras el arranque.

```

Shell> pci -b
  Seg  Bus  Dev  Func
  ---  ---  ---  ----
    00   00   00   00 ==> Bridge Device - Host/PCI bridge
        Vendor 8086 Device 1237 Prog Interface 0
    00   00   01   00 ==> Bridge Device - PCI/ISA bridge
        Vendor 8086 Device 7000 Prog Interface 0
    00   00   01   01 ==> Mass Storage Controller - IDE controller
        Vendor 8086 Device 7010 Prog Interface 80
    00   00   01   03 ==> Bridge Device - Other bridge type
        Vendor 8086 Device 7113 Prog Interface 0
    00   00   02   00 ==> Display Controller - VGA/8514 controller
        Vendor 1234 Device 1111 Prog Interface 0

```

2.8: output comando pci -b

```

D          Y C I
R          P F A
U  VERSION E G G #D #C DRIVER NAME                IMAGE NAME
== ===== = = = == == =====
59 0000000A B - - 1 5 PCI Bus Driver                PciBusDxe
5B 00000010 ? - - - - Virtio PCI Driver            VirtioPciDeviceDxe
5C 00000010 ? - - - - Virtio 1.0 PCI Driver        Virtio10
5D 00000010 ? - - - - Virtio Block Driver          VirtioBlkDxe
5E 00000010 ? - - - - Virtio SCSI Host Driver        VirtioScsiDxe
5F 00000010 ? - - - - Virtio Random Number Generator Driu VirtioRngDxe
60 00000010 ? - - - - Virtio Serial Driver          VirtioSerialDxe
61 0000000A D - - 2 - Platform Console Management Driver ConPlatformDxe
62 0000000A D - - 2 - Platform Console Management Driver ConPlatformDxe
63 0000000A B - - 2 2 Console Splitter Driver        ConSplitterDxe
64 0000000A ? - - - - Console Splitter Driver        ConSplitterDxe
65 0000000A ? - - - - Console Splitter Driver        ConSplitterDxe
66 0000000A B - - 2 2 Console Splitter Driver        ConSplitterDxe
67 0000000A B - - 1 1 Console Splitter Driver        ConSplitterDxe
6B 0000000A D - - 1 - Graphics Console Driver        GraphicsConsoleDxe
6C 0000000A B - - 1 1 Serial Terminal Driver        TerminalDxe
6D 0000000A D - - 3 - Generic Disk I/O Driver        DiskIoDxe
6E 0000000B B - - 1 1 Partition Driver (MBR/GPT/El Torito) PartitionDxe
71 0000000A B - - 1 1 SCSI Bus Driver                ScsiBus
72 0000000A D - - 1 - Scsi Disk Driver                ScsiDisk
73 0000000A D - - 1 - Sata Controller Init Driver        SataController
74 00000010 D - - 1 - AtaAtapiPassThru Driver        AtaAtapiPassThruDxe
75 00000010 B - - 1 1 ATA Bus Driver                AtaBusDxe
76 00000010 ? - - - - NUM Express Driver            NumExpressDxe
77 00000010 B - - 1 3 UMF Sio Bus Driver            SioBusDxe

```

2.9: output comando drivers -b

3 Trabajo Práctico 2:

Desarrollo, compilación y análisis de seguridad

3.1 Desarrollo de la Aplicación

La aplicación desarrollada tiene un objetivo deliberadamente mínimo: validar la ejecución de código nativo dentro del entorno UEFI y dejar una evidencia clara tanto en la salida de consola como en el análisis estático posterior. Para ello se inicializa la biblioteca de GNU-EFI, se imprime un mensaje inicial y luego se define un byte con valor `0xCC`, opcode que corresponde a la instrucción `INT3`, ampliamente utilizada como breakpoint de software.

La función de entrada `efi_main` sigue la convención esperada para una aplicación UEFI: recibe un `EFI_HANDLE` que identifica la imagen cargada y un puntero a `EFI_SYSTEM_TABLE`, estructura central que da acceso a los servicios del firmware. Desde esa tabla se alcanzan, entre otros, los punteros a `Boot Services`, `Runtime Services` y a los protocolos básicos de entrada/salida de consola. La llamada a `InitializeLib` inicializa el soporte de GNU-EFI para simplificar el acceso a esas estructuras.

```
1  #include <efi.h>
2  #include <efilib.h>
3
4  EFI_STATUS efi_main(EFI_HANDLE ImageHandle, EFI_SYSTEM_TABLE *SystemTable)
5      InitializeLib(ImageHandle, SystemTable);
6      SystemTable->ConOut->OutputString(SystemTable->ConOut, L"Iniciando ana
7
8      // Inyección de un software breakpoint (INT3)
9      unsigned char code[] = { 0xCC };
10
11     if (code[0] == 0xCC) {
12         SystemTable->ConOut->OutputString(SystemTable->ConOut, L"Breakpoin
13     }
14
15     return EFI_SUCCESS;
16 }
```

El uso de una condición sobre `0xCC` permite que el valor sea visible al descompilar el binario, facilitando la correlación entre el código fuente, el ensamblado y el pseudocódigo generado por la herramienta de análisis.

Desde el punto de vista didáctico, este ejemplo también sirve para observar que el firmware ejecuta aplicaciones en un contexto mucho más cercano al hardware que una aplicación de usuario tradicional. No hay aislamiento provisto por un kernel, ni loader de espacio de usuario, ni libc completa por defecto; por ello cada interacción con pantalla, variables o memoria debe realizarse mediante primitivas expuestas por la propia especificación UEFI.

Pregunta: *¿Por qué utilizamos `SystemTable->ConOut->OutputString` en lugar de la función `printf` de C?*

Se utiliza por que en ese momento nos encontramos en el entorno de UEFI donde no se han cargado las librerías estándar de C y por lo tanto debemos usar protocolos de salida de la `SystemTable` de UEFI.

En particular, `ConOut` implementa el protocolo de salida de texto simple del firmware. `OutputString` espera cadenas en formato ancho (`CHAR16`), de allí el uso del prefijo `L` en los literales. Esta diferencia de representación de cadenas y la ausencia de un entorno POSIX completo dejan en claro que una aplicación UEFI no puede asumir la infraestructura típica de un proceso de Linux o Windows en espacio de usuario.

3.2 Compilación a Formato PE/COFF

La compilación se realizó en tres etapas. Primero se generó un objeto relocatable a partir del código fuente con opciones compatibles con un entorno freestanding. Luego se enlazó dicho objeto con las bibliotecas de GNU-EFI para producir un binario intermedio. Finalmente, `objcopy` convirtió el resultado al formato EFI `application`, que internamente corresponde a PE/COFF, formato esperado por el firmware UEFI.

Las flags empleadas en la compilación no son arbitrarias. `-ffreestanding` indica que el programa no debe compilarse asumiendo un entorno hospedado por un sistema operativo; `-fshort-wchar` ajusta el tamaño de `wchar_t` al formato usado por EFI; `-mno-red-zone` evita el uso de la red zone del ABI System V, ya que no es seguro asumir su disponibilidad en este contexto; y `-fno-stack-protector` elimina dependencias con mecanismos de protección de pila que requerirían soporte adicional en tiempo de ejecución.

```

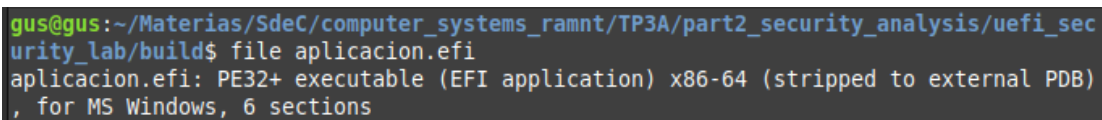
1  # 1. Compilar a código objeto
2  gcc -I/usr/include/efi -I/usr/include/efi/x86_64 -I/usr/include/efi/protoc
3      -fpic -ffreestanding -fno-stack-protector -fno-strict-aliasing \
4      -fshort-wchar -mno-red-zone -maccumulate-outgoing-args -Wall \
5      -c -o aplicacion.o aplicacion.c
6
7  # 2. Linkear a un binario intermedio
8  ld -shared -Bsymbolic -L/usr/lib -L/usr/lib/efi \
9      -T /usr/lib/elf_x86_64_efi.lds /usr/lib/crt0-efi-x86_64.o \
10     aplicacion.o -o aplicacion.so -lefi -lgnuefi
11
12 # 3. Convertir al ejecutable EFI final
13 objcopy -j .text -j .sdata -j .data -j .dynamic -j .dynsym \
14         -j .rel -j .rela -j .rel.* -j .rela.* -j .reloc \
15         --target=efi-app-x86_64 aplicacion.so aplicacion.efi

```

Esta secuencia muestra una diferencia importante respecto de una aplicación de espacio de usuario tradicional: aunque el desarrollo se realiza desde Linux, el artefacto final no es un ELF ejecutable para el sistema operativo anfitrión, sino un binario PE32+ preparado para ser cargado directamente por el firmware.

También es relevante notar que el uso de un `.so` intermedio no implica que se esté construyendo una biblioteca compartida para Linux en el sentido convencional. Ese artefacto funciona como paso técnico dentro del pipeline de GNU-EFI para ordenar secciones, resolver símbolos y preparar las relocalizaciones necesarias antes de la conversión final a imagen EFI.

3.3 Análisis de Metadatos y Decompilación



```

gus@gus:~/Materias/SdeC/computer_systems_ramnt/TP3A/part2_security_analysis/uefi_security_lab/build$ file aplicacion.efi
aplicacion.efi: PE32+ executable (EFI application) x86-64 (stripped to external PDB)
, for MS Windows, 6 sections

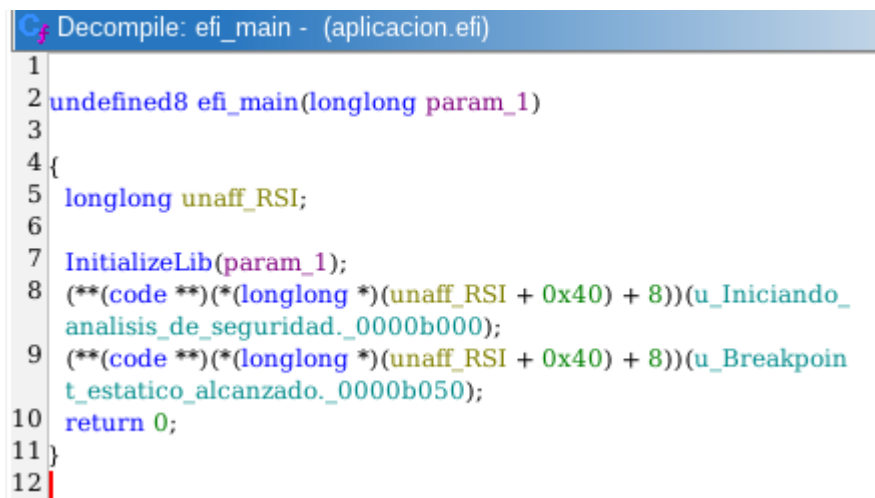
```

3.1: verificación del tipo de archivo de `aplicacion.efi`

La inspección del archivo resultante confirma que `aplicacion.efi` fue generado como PE32+ executable (EFI application) x86-64, lo cual es consistente con la especificación de UEFI. Esto valida que la conversión final fue correcta y que el binario ya no debe interpretarse como un ejecutable ELF convencional.

En consecuencia, herramientas orientadas exclusivamente a ELF no resultan adecuadas para extraer todos sus metadatos. El dato relevante para esta práctica es que el firmware espera un ejecutable PE/COFF y que la herramienta `file` lo reconoce explícitamente como una aplicación EFI.

La elección de PE32+ para arquitecturas de 64 bits responde a la compatibilidad histórica de UEFI con el ecosistema PE/COFF y a la necesidad de contar con un formato bien definido para secciones, cabeceras y relocación. Que el binario sea x86-64 además debe coincidir con la arquitectura del firmware virtual utilizado; un desajuste entre arquitectura de la imagen y del firmware impediría su carga.



```

C: Decompiler: efi_main - (aplicacion.efi)
1
2 undefined8 efi_main(longlong param_1)
3
4 {
5     longlong unaff_RSI;
6
7     InitializeLib(param_1);
8     (**(code **)(*(longlong *)(unaff_RSI + 0x40) + 8))(u_Iniciando_
    analisis_de_seguridad._0000b000);
9     (**(code **)(*(longlong *)(unaff_RSI + 0x40) + 8))(u_Breakpoint
    t_estatico_alcanzado._0000b050);
10    return 0;
11 }
12

```

3.2: pseudocódigo de `efi_main` en Ghidra

00003064	48 8d 0d 95	LEA	param_1,[u_Iniciando_analisis_de_seguridad._0000b...
	7f 00 00		
0000306b	48 89 ce	MOV	RSI,param_1
0000306e	48 89 c7	MOV	RDI,RAX
00003071	ff d2	CALL	RDX
00003073	c6 45 ff cc	MOV	byte ptr [RBP + local_9],0xcc
00003077	0f b6 45 ff	MOVZX	EAX,byte ptr [RBP + local_9]
0000307b	3c cc	CMP	AL,0xcc
0000307d	75 23	JNZ	LAB_000030a2
0000307f	48 8b 45 e0	MOV	RAX,qword ptr [RBP + local_28]
00003083	48 8b 40 40	MOV	RAX,qword ptr [RAX + 0x40]
00003087	48 8b 50 08	MOV	RDX,qword ptr [RAX + 0x8]
0000308b	48 8b 45 e0	MOV	RAX,qword ptr [RBP + local_28]
0000308f	48 8b 40 40	MOV	RAX,qword ptr [RAX + 0x40]
00003093	48 8d 0d b6	LEA	param_1,[u_Breakpoint_estatico_alcanzado._0000b0...

3.3: desensamblado asociado a la comparacion con `0xCC`

En Ghidra se observa que la función `efi_main` conserva la estructura esencial del programa: inicialización de la biblioteca, llamada al método `OutputString` de `ConOut` e inspección del byte almacenado localmente antes de imprimir el segundo mensaje. El

desensamblado también permite ubicar la comparación contra `0xCC`, lo que conecta el comportamiento esperado en tiempo de ejecución con su representación a bajo nivel.

En el pseudocódigo puede verse además que las llamadas a los servicios de salida no aparecen como una invocación directa a una función de C estándar, sino como desreferenciaciones de punteros a funciones obtenidos desde estructuras del firmware. Esa observación es importante porque refleja con fidelidad el modelo de programación de UEFI: la interacción con el entorno se realiza a través de tablas y protocolos, no mediante syscalls ni bibliotecas del sistema operativo.

Pregunta: *En el pseudocódigo de Ghidra, la condición `0xCC` suele aparecer como `-52`. ¿A qué se debe este fenómeno y por qué importa en ciberseguridad?*

Aparece como `-52` por una interpretación binaria al ser `1100 1100` se interpreta en complemento a dos, es decir, el 1 más significativo nos indica que es negativo, luego se invierten los bits y se suma 1 para obtener el valor absoluto, haciendo esto se consigue `0011 0100` que es igual a 52. Es importante en ciberseguridad dado que representa la INT 3, que es la instrucción en el lenguaje ensamblador x86/x64 que realiza una pausa en la ejecución del programa y genera un “breakpoint exception”, esto permite realizar diversas inspecciones. Ej.: En los exploits por buffer overflow `0xCC` se usa a menudo como un marcador para rastrear el flujo de ejecución o asegurar que el código llegue al lugar correcto.

Más formalmente, el valor `0xCC` puede ser mostrado por un descompilador como entero con signo de 8 bits, cuyo rango es `-128` a `127`. En ese dominio, `0xCC` equivale a `204` sin signo, pero a `-52` con signo. Comprender esta diferencia es relevante al auditar binarios, ya que una lectura incorrecta del tipo de dato puede hacer que un analista pase por alto constantes significativas, marcadores de depuración o secuencias de instrucciones con valor semántico en una investigación de seguridad.

4 Trabajo Práctico 3:

Ejecución en Hardware Físico (Bare Metal)

4.1 Preparación del medio de arranque (USB)

En esta etapa el objetivo es trasladar al hardware físico el binario generado en la parte 2, respetando los requisitos de arranque de la especificación UEFI. Para ello se utiliza un pendrive formateado en FAT32, ya que este sistema de archivos es el contemplado por la especificación para la partición del sistema EFI y permite que el firmware localice de forma estándar ejecutables `.efi`.

La estrategia consiste en preparar el dispositivo USB con dos elementos principales: una shell UEFI en la ruta por defecto `EFI/BOOT/BOOTX64.EFI`, que será utilizada como primer punto de entrada, y la aplicación `aplicacion.efi` copiada a la raíz del dispositivo para su ejecución manual desde dicha shell. Este enfoque evita depender de un sistema operativo instalado en el equipo y permite validar el comportamiento del binario directamente en el contexto del firmware real.

```
1  # 1. Identificar el dispositivo USB
2  lsblk
3
4  # 2. Formatear en FAT32
5  sudo mkfs.vfat -F 32 /dev/sdb1
6
7  # 3. Montar el pendrive
8  sudo mount /dev/sdb1 /mnt
9
10 # 4. Crear la estructura de arranque UEFI estándar
11 sudo mkdir -p /mnt/EFI/BOOT
12
13 # 5. Descargar la shell UEFI oficial de Tianocore
14 sudo wget https://github.com/tianocore/edk2/raw/UDK2018/ShellBinPkg/UefiSh
15      -O /mnt/EFI/BOOT/BOOTX64.EFI
```



```
17 # 6. Copiar la aplicación compilada en el TP2
18 sudo cp ../part2_development_compilation_security_analysis/uefi_security_1
19
20 # 7. Sincronizar y desmontar
21 sudo sync
22 sudo umount /mnt
```

La ubicación `EFI/BOOT/BOOTX64.EFI` no es arbitraria: corresponde a la ruta de fallback definida por UEFI para medios removibles de 64 bits. Si el firmware no encuentra una entrada de arranque previamente registrada en NVRAM para ese medio, puede igualmente localizar esa ruta predeterminada e iniciar la shell allí ubicada.

4.2 Configuración del BIOS/UEFI

Una vez preparado el dispositivo, fue necesario ajustar la configuración del firmware de la computadora física para permitir la ejecución del entorno de prueba. En particular, se deshabilitó `Secure Boot`, ya que tanto la shell descargada como la aplicación generada durante el trabajo práctico no están firmadas por una autoridad reconocida por el firmware del fabricante.

Desde el punto de vista de seguridad, `Secure Boot` actúa como un mecanismo de validación criptográfica de la cadena de arranque: solo permite cargar binarios EFI cuya firma pueda verificarse contra claves confiables almacenadas en firmware. Esto protege contra bootkits y cargadores maliciosos, pero también bloquea cualquier binario experimental o académico que no haya sido firmado adecuadamente. Por eso, para esta práctica, fue necesario desactivar temporalmente dicha protección.

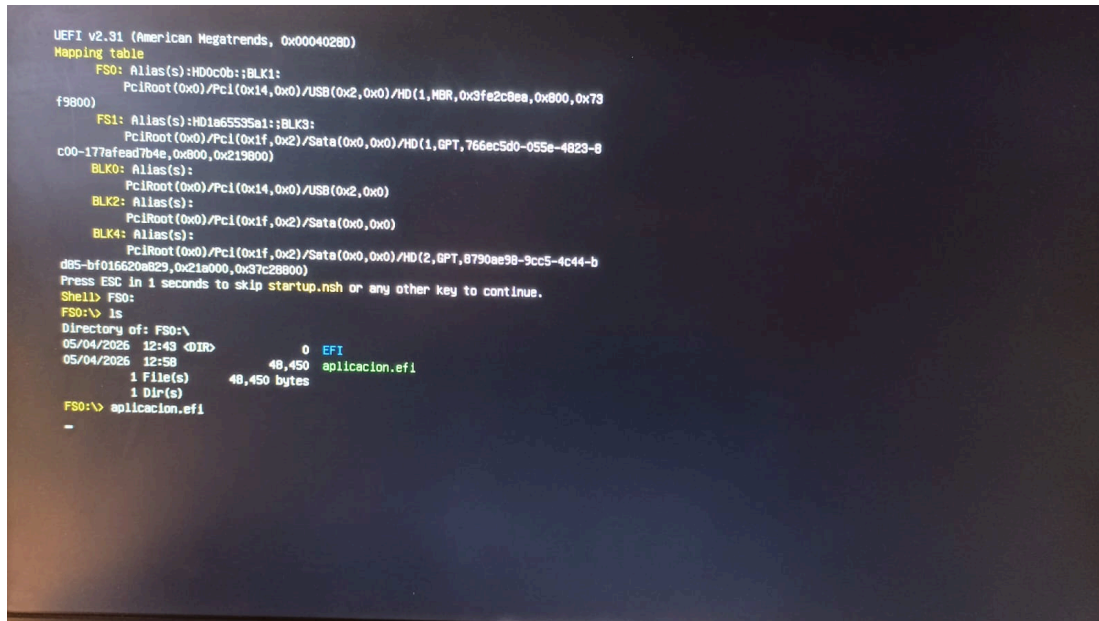
4.3 Ejecución en Bare Metal

Con el pendrive preparado y la configuración del firmware ajustada, la ejecución en bare metal consiste en arrancar el equipo desde el medio removible, ingresar a la shell UEFI y lanzar manualmente la aplicación compilada. A diferencia del entorno emulado, aquí el binario se ejecuta sobre el firmware y el hardware reales del equipo, lo que permite verificar compatibilidad práctica con la implementación UEFI del fabricante.

```
1 Shell> FS0:
2 FS0:\> ls
```

Una vez realizada la ejecución de los comandos propuestos, obtuvimos el siguiente comportamiento (observado en la siguiente captura):

Si la ejecución es correcta, el firmware debe transferir el control al punto de entrada `efi_main` de la imagen cargada y la salida debe imprimirse a través del protocolo de consola del sistema, sin intervención de un kernel ni de un sistema operativo anfitrión. Esto demuestra que el binario construido en la parte 2 no solo posee el formato correcto, sino que también es funcional en un entorno UEFI real.



```
UEFI v2.31 (American Megatrends, 0x00040280)
Mapping table
FS0: Alias(s):HD0c0b:;BLK1:
    PciRoot(0x0)/Pci(0x14,0x0)/USB(0x2,0x0)/HD(1,MBR,0x3fe2c8e,0x800,0x79
f900)
FS1: Alias(s):HD1a65535a1:;BLK3:
    PciRoot(0x0)/Pci(0x1f,0x2)/Sata(0x0,0x0)/HD(1,GPT,766ec5d0-055e-4823-8
c00-177afead7b4e,0x800,0x21900)
BLK0: Alias(s):
    PciRoot(0x0)/Pci(0x14,0x0)/USB(0x2,0x0)
BLK2: Alias(s):
    PciRoot(0x0)/Pci(0x1f,0x2)/Sata(0x0,0x0)
BLK4: Alias(s):
    PciRoot(0x0)/Pci(0x1f,0x2)/Sata(0x0,0x0)/HD(2,GPT,8790ae98-9cc5-4c44-b
d85-bf016620a829,0x21e000,0x37c2800)
Press ESC in 1 seconds to skip startup.nsh or any other key to continue.
Shell> FS0:
FS0:\> ls
Directory of: FS0:\
05/04/2026 12:43 DIR          0 EFI
05/04/2026 12:58      48,450 aplicacion.efi
1 File(s)          48,450 bytes
1 Dir(s)
FS0:\> aplicacion.efi
-
```

4.1: Salida de la shell UEFI al ejecutar aplicacion.efi

Texto esperado en pantalla:

- 1 Iniciando analisis de seguridad...
- 2 Breakpoint estatico alcanzado.

La realidad es que al ejecutar `aplicacion.efi` en el hardware físico, la shell UEFI no mostró la salida esperada y se mantuvo sin respuesta. Esto sugiere que, aunque el binario fue reconocido y ejecutado, no logró interactuar correctamente con los servicios de salida del firmware o que hubo un error en la ejecución que impidió que el mensaje se imprimiera.

Pregunta: *¿Por qué fue necesario deshabilitar Secure Boot para ejecutar la shell UEFI y la aplicación desarrollada en la práctica?*

firmware. Tanto la shell UEFI utilizada como `aplicacion.efi` son artefactos de laboratorio y no cuentan con una firma reconocida por el fabricante del equipo, por lo que el firmware los rechazaría con un error de violación de seguridad si esa protección permaneciera activa.

Desde una perspectiva técnica, esto confirma que **Secure Boot** no evalúa si el binario “parece legítimo” por su nombre o ubicación, sino si la imagen PE/COFF contiene una firma válida encadenable a una base de confianza instalada en la plataforma. En consecuencia, la práctica también evidencia la tensión entre flexibilidad de experimentación y garantías de integridad en la cadena de arranque.

5 Conclusión

El desarrollo de este trabajo práctico permitió recorrer de forma progresiva las principales capas del entorno UEFI, desde la exploración inicial de sus abstracciones de hardware hasta la compilación y ejecución de una aplicación nativa fuera de un sistema operativo convencional. A lo largo de la práctica se observó que UEFI no debe entenderse únicamente como un reemplazo moderno del BIOS, sino como una plataforma de firmware con servicios, protocolos y estructuras propias, capaces de ofrecer un entorno de ejecución relativamente estandarizado en la etapa de prearranque.

En la primera parte, el análisis de handles, protocolos, variables de NVRAM y mapas de memoria permitió comprender cómo el firmware organiza los recursos del sistema y cómo define la secuencia de arranque a través de políticas persistentes. En la segunda parte, la construcción de una aplicación en C y su transformación al formato PE/COFF mostró las particularidades del desarrollo de software UEFI, donde no existe una infraestructura equivalente a la de un proceso en espacio de usuario y toda interacción debe canalizarse mediante servicios del propio firmware. Finalmente, la tercera parte permitió trasladar ese conocimiento al hardware físico y evidenciar que la ejecución bare metal introduce restricciones y comportamientos adicionales, tales como la necesidad de deshabilitar **Secure Boot** y las diferencias prácticas entre un entorno emulado y la implementación concreta del firmware de un fabricante.

El resultado general de la experiencia no solo aporta una comprensión más precisa del flujo de arranque y del formato de los binarios EFI, sino también una visión relevante desde la seguridad ofensiva y defensiva. El estudio de regiones persistentes de memoria, variables globales y mecanismos como **Secure Boot** muestra con claridad por qué UEFI constituye un punto crítico dentro de la cadena de confianza del sistema. En consecuencia, este trabajo no solo cumple un objetivo de familiarización técnica, sino que también sienta una base sólida para analizar riesgos, capacidades de persistencia y mecanismos de protección en plataformas modernas.

6 Conclusion – Respuestas solicitadas

¿Cuál es la ventaja de seguridad y compatibilidad del modelo de handles y protocolos de UEFI frente al antiguo BIOS?

La principal ventaja es que este modelo desacopla al software de prearranque de los detalles concretos del hardware. En lugar de acceder directamente a puertos o registros específicos, las aplicaciones interactúan con protocolos expuestos por el firmware, lo que mejora la compatibilidad entre plataformas y reduce la dependencia de una arquitectura particular.

Desde el punto de vista de seguridad, esto también reduce la superficie de ataque. Al obligar a que el acceso al hardware pase por interfaces controladas por el firmware, se limita la posibilidad de que código malicioso manipule periféricos de forma directa o no autorizada. A su vez, el mismo binario puede ser reutilizado sobre distintos equipos siempre que el firmware implemente los protocolos definidos por la especificación UEFI.

¿Cómo determina el Boot Manager la secuencia de arranque a partir de `Boot####` y `BootOrder`?

La secuencia de arranque es determinada por la variable global `BootOrder`, que contiene una lista ordenada de identificadores `Boot####`. Cada entrada `Boot####` describe una opción concreta de arranque, incluyendo atributos, descripción visible, dispositivo asociado y la ruta al ejecutable EFI que debe cargarse.

El Boot Manager recorre esa lista en el orden definido por `BootOrder` e intenta ejecutar cada opción sucesivamente. Si una entrada falla o no está disponible, continúa con la siguiente. De esta forma, `BootOrder` define la política de prioridad, mientras que `Boot####` representa el conjunto de destinos de arranque posibles.

¿Por qué las regiones `RuntimeServicesCode` son un objetivo principal para malware tipo Bootkit?

Estas regiones son especialmente sensibles porque permanecen activas en memoria incluso después de que el sistema operativo haya terminado de arrancar. A diferencia de `BootServicesCode`, que deja de ser válida tras la transición al sistema operativo,

`RuntimeServicesCode` sigue siendo utilizada para servicios como acceso a variables EFI, consulta de tiempo o reinicios controlados.

Si un atacante consigue introducir código malicioso en esas zonas, puede interceptar llamadas entre el sistema operativo y el firmware con un nivel muy alto de persistencia y privilegio. Esto convierte a esas regiones en un punto de interés crítico para bootkits y otras amenazas que buscan mantenerse ocultas por debajo del sistema operativo.

¿Por qué se utiliza `SystemTable->ConOut->OutputString` en lugar de `printf` en una aplicación UEFI?

Se utiliza `SystemTable->ConOut->OutputString` porque, en el entorno UEFI, todavía no existe una biblioteca estándar de C equivalente a la disponible en un proceso convencional de sistema operativo. La aplicación se ejecuta en un contexto freestanding y debe interactuar con la salida mediante los servicios que provee directamente el firmware.

En particular, `ConOut` implementa el protocolo de salida de texto simple de UEFI, y `OutputString` permite imprimir cadenas en formato `CHAR16`. Esto deja en claro que una aplicación UEFI no puede asumir la existencia de `printf`, POSIX ni un runtime completo de usuario, sino que debe trabajar con las abstracciones propias del firmware.

¿Por qué Ghidra muestra `0xCC` como `-52` y por qué importa en ciberseguridad?

Esto sucede porque el valor `0xCC` puede ser interpretado como un entero con signo de 8 bits. En ese dominio, `0xCC` equivale a 204 sin signo, pero a `-52` cuando se interpreta en complemento a dos. Por eso el descompilador puede mostrar esa constante con signo aunque en el código fuente se haya pensado como un byte hexadecimal.

La cuestión es importante en ciberseguridad porque `0xCC` corresponde a la instrucción `INT3`, utilizada como breakpoint de software en arquitecturas x86/x64. Reconocer correctamente ese valor al analizar binarios permite identificar marcadores de depuración, trazas de explotación o secuencias relevantes para la inspección del flujo de ejecución.

¿Por qué fue necesario deshabilitar Secure Boot para ejecutar la shell UEFI y la aplicación del trabajo práctico?

Fue necesario deshabilitar `Secure Boot` porque este mecanismo solo permite la ejecución de binarios EFI cuya firma sea válida frente a una base de confianza almacenada en el

firmware. Tanto la shell UEFI utilizada como `aplicacion.efi` fueron artefactos de laboratorio y no contaban con una firma reconocida por el fabricante del equipo.

Si **Secure Boot** hubiera permanecido activo, el firmware habría rechazado la carga de esos binarios con un error de violación de seguridad. Esto muestra con claridad que **Secure Boot** protege la cadena de arranque verificando la integridad y autenticidad criptográfica de las imágenes PE/COFF, aunque al mismo tiempo limita la ejecución de binarios experimentales no firmados.